PHYS562 Assignment 3:
FDTD Simulation

Robert Pascua

(Dated: March 9, 2021)

I. INTRODUCTION

This document gives a brief overview of the finite-difference time-domain (FDTD) simulation performed for Assignment 3. The simulation was performed on a two-dimensional Yee lattice for TE_z-polarized fields, used a perfectly matched layer (PML) absorbing boundary condition, and allowed for inhomogenous linear media to be placed in the non-absorbing section of the simulation domain. The simulation was implemented in Python and requires the following dependencies: `cached_property`[1], `numpy` [3][2], and `astropy`[1][3]. The relevant code is available on GitHub via: `https://github.com/r-pascua/phys562`; there is an exploratory notebook and some basic code available for viewing. This document is organized as follows: Section II describes how the simulation is initialized; Section III describes how the fields are advanced and provides additional derivations to help understand the implementation; Section IV describes the various simulations performed and presents some results; Section V provides conclusions about the project and discusses future work to be done.

II. SIMULATION SETUP

Since this simulation was designed for simulating the evolution of TE_z-polarized fields in two dimensions, the only relevant fields are E_x, E_y, and H_z. Only three parameters are required to initialize a simulation object: `x_cells`, which specifies the number of cells along the x-axis in the non-absorbing region; `y_cells`, which specifies the number of cells along the y-axis in the non-absorbing region; and `pml_width`, which specifies the number of cells to use in each PML. The simulation object currently only supports simulating a uniform space that is surrounded on each side by a PML, with each PML having the same width as every other PML. Provided these parameters, a mesh of shape (M, N) is constructed, where `M = x_cells + 2 * pml_width + 1` and `N = y_cells + 2 * pml_width + 1`, so that the mesh consists of points at the corner of every cell in the simulation region. Three subsequent meshes are constructed that map to the locations of the fields on the Yee lattice: `Ex_mesh`, which has shape $(M-1, N)$; `Ey_mesh`, which has shape $(M, N-1)$; and `Hz_mesh`, which has shape $(M-1, N-1)$. These meshes are used for determining different coefficients in the integration scheme and are used (along with a specification of the number of time steps in

[1] `https://pypi.org/project/cached-property/`

[2] `https://numpy.org/`

[3] `https://www.astropy.org/index.html`

the simulation) to construct three-dimensional arrays that store the fields: `Ex`, `Ey`, `Hz`. Using the usual Yee lattice notation and advancing the fields in time under a leapfrog scheme, the following correspondences exist between the arrays in memory and the physical fields they represent:

$$\texttt{Ex[n,i,j]} \rightarrow E^{n+1/2}_{x,(i+1/2)j} \tag{1}$$

$$\texttt{Ey[n,i,j]} \rightarrow E^{n+1/2}_{y,i(j+1/2)} \tag{2}$$

$$\texttt{Hz[n,i,j]} \rightarrow H^{n}_{z,(i+1/2)(j+1/2)}. \tag{3}$$

Note that although only one magnetic field is specified above, the actual implementation treats the fields as split everywhere—that is, instead of a single `Hz`, there are two arrays `Hzx` and `Hzy` that obey `Hz = Hzx + Hzy`. This implementation is not as memory efficient as it could be, but this decision was made in the interest of code readability and simplicity from a development perspective in the context of employing Berenger's PML as an absorbing boundary.

In addition to the bare-bones setup described above, the user may specify additional parameters when initializing a simulation object. Additional parameters are detailed as follows: `spatial_resolution` gives the physical length of one side of a simulation cell (in meters), with the default value being one centimeter (since this is primarily intended to be used for studying radio frequency phenomena); `courant_number` is the Courant number that provides the relationship between the length of a single integration and the size of a cell, with the default value being $1/\sqrt{2}$ (corresponding to a wavefront traversing a cell diagonally in two time steps); `Ntimes` defines the number of integrations to perform when running the simulation, but it may be overwritten by specifying this parameter when running a simulation; `relative_permittivity` is either a scalar (for uniform permittivity) or an appropriately-sized array[4] specifying ε_r, where $\varepsilon = \varepsilon_0\varepsilon_r$; `relative_permeability` is similarly defined[5] and specifies μ_r, where $\mu = \mu_0\mu_r$; `pml_conductivity_scale` defines the maximum value of the PML conductivity, which is attained at the outer edge of the PML (it is σ_m in Equation 35 of Berenger [2]); `pml_scaling_index` defines the power-law index in the PML conductivity profile (it is n in the same equation from Berenger [2]).

[4] See the documentation in the code for details on how to specify the permittivity using an array.

[5] The requirements for passing an array are different than they are for the permittivity; see the documentation in the code for details.

III. TIME-EVOLUTION

In accordance with usual FDTD schemes, the electric and magnetic fields are advanced according to Faraday's Law and the Ampère-Maxwell Law. In two dimensions, these equations take the following form (using Berenger's split field formalism):

$$\varepsilon \partial_t E_x = -\sigma E_x + \partial_y H_z, \tag{4}$$

$$\varepsilon \partial_t E_y = -\sigma E_y - \partial_x H_z, \tag{5}$$

$$\mu \partial_t H_{zx} = -\sigma^* H_{zx} - \partial_x E_y, \tag{6}$$

$$\mu \partial_t H_{zy} = -\sigma^* H_{zy} + \partial_y E_x, \tag{7}$$

where μ is the permeability, ε is the permittivity, σ is the electric conductivity, σ^* is the magnetic conductivity, and $H_z = H_{zx} + H_{zy}$. These equations can be discretized to give the update equations in the non-absorbing region:

$$E^{n+1/2}_{x,(i+1/2)j} = E^{n-1/2}_{x,(i+1/2)j} + \frac{\Delta t}{\varepsilon_{(i+1/2)j}\delta}\Big(H^n_{z,(i+1/2)(j+1/2)} - H^n_{z,(i+1/2)(j-1/2)}\Big), \tag{8}$$

$$E^{n+1/2}_{y,i(j+1/2)} = E^{n-1/2}_{y,i(j+1/2)} - \frac{\Delta t}{\varepsilon_{i(j+1/2)}\delta}\Big(H^n_{z,(i+1/2)(j+1/2)} - H^n_{z,(i-1/2)(j+1/2)}\Big), \tag{9}$$

$$H^{n+1}_{zx,(i+1/2)(j+1/2)} = H^n_{zx,(i+1/2)(j+1/2)} - \frac{\Delta t}{\mu_{(i+1/2)(j+1/2)}\delta}\Big(E^{n+1/2}_{y,i(j+1/2)} - E^{n+1/2}_{y,(i-1)(j+1/2)}\Big), \tag{10}$$

$$H^{n+1}_{zy,(i+1/2)(j+1/2)} = H^n_{zy,(i+1/2)(j+1/2)} + \frac{\Delta t}{\mu_{(i+1/2)(j+1/2)}\delta}\Big(E^{n+1/2}_{x,(i+1/2)j} - E^{n+1/2}_{x,(i+1/2)(j-1)}\Big). \tag{11}$$

The update equations in the PML are adapted according to Equations 37 and 38 in Berenger [2]. Following the discussions in Schneider [4], the dependence of these equations on the integration time Δt and spatial resolution δ can be entirely removed in the non-absorbing region by noting the relation between these parameters and the Courant number S_c:

$$S_c \equiv \frac{c\Delta t}{\delta}. \tag{12}$$

Noting that $c = 1/\sqrt{\varepsilon_0 \mu_0}$ and defining the impedance of free space as $\eta_0 = \sqrt{\mu_0/\varepsilon_0}$, the pre-factors on the finite differences (assuming linear media and omitting the indices on the relative permeability and relative permittivity) can be written as:

$$\frac{\Delta t}{\varepsilon\delta} = \frac{S_c \eta_0}{\epsilon_r} \tag{13}$$

$$\frac{\Delta t}{\mu\delta} = \frac{S_c}{\eta_0 \mu_r}. \tag{14}$$

In addition to applying this substitution in Equations 8-11, the following substitutions are also made wherever they appear in the update equations for the PML:

$$\frac{\Delta t}{\varepsilon} = \frac{S_c \delta}{\varepsilon_r} \eta_0, \tag{15}$$

$$\frac{\Delta t}{\mu} = \frac{S_c \delta}{\eta_0 \varepsilon_r}, \tag{16}$$

$$\sigma^* = \frac{\mu_r}{\varepsilon_r} \sigma_0^*, \tag{17}$$

where σ_0^* is the magnetic conductivity for a layer perfectly matched to free space, provided an electric conductivity. Using these substitutions, we never actually need to know the physical integration time—all that's necessary is the physical length of the side of one cell and the Courant number.

The integration scheme is performed in the usual leapfrog manner: the magnetic field is updated in accordance with the update equations, the magnetic field source is applied, and then the electric field is updated, and this process is repeated until the desired number of integrations have been completed. Since memory limitations are not an issue for two-dimensional problems implemented on modern computers, the entirety of the simulation history is stored in memory; however, this is certainly not necessary, as one only needs to know the fields at two times for each point in space in order to advance the fields. In order to minimize computation time, the various multiplicative factors in the update equations are calculated once and cached for retrieval each time they are needed. Additionally, in an effort to make the update code as simple as possible, these factors are stored at every point in space; examples are shown in Figure 1. This is sufficient for a problem involving linear media, but will need to be modified if this program is extended to deal with nonlinear media. Once a simulation object (an instance of the `Simulator` class) has been created, a simulation can be run by using the `simulate` method with some source parameters provided that specify the source function, the source's location, and any additional parameters needed for defining the source function. The `Simulator` class has some built-in source functions available that can be used by specifying the source function with an appropriate text string; otherwise, a custom source can be provided as a callable function, provided it has the appropriate signature. For further details on the sources available, the syntax required for providing custom sources, and how to specify the source location, please refer to the documentation in the code.

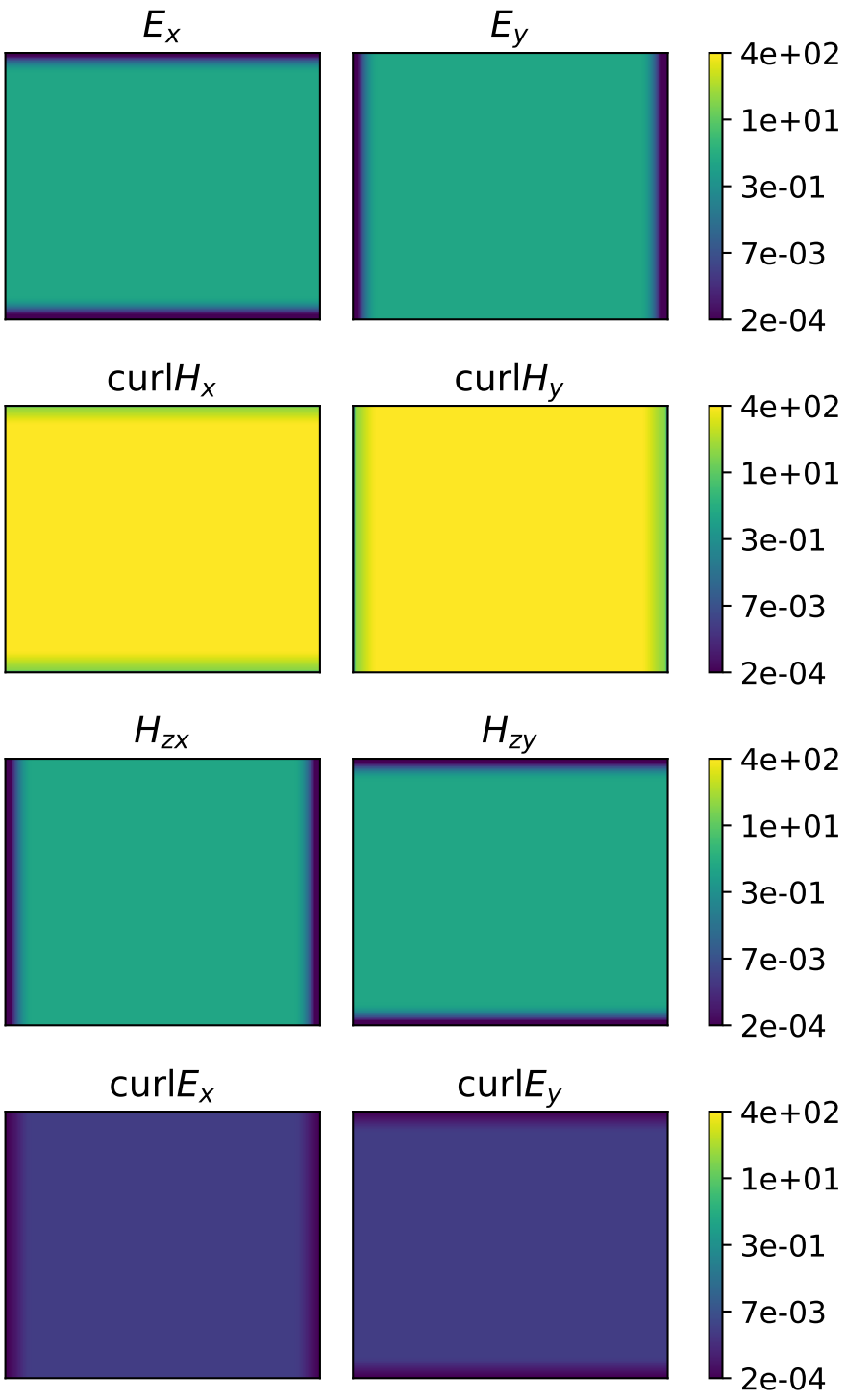

FIG. 1. Numerical factors multiplying the various quantities used in advancing the fields. The titles give the corresponding quantity, with subscripts on the curl values denoting the component of the curl. This is for a simulation box with free space in the non-absorbing region, a 10-cell thick PML layer, and default PML parameters.

IV. SIMULATION RESULTS

A variety of test simulations were performed, as well as some qualitative simulations intended to experiment with the FDTD developed here. Unfortunately, there seems to be some sort of bug in my code that I cannot uncover, since the fields apparently increase without bound; however, their relative amplitudes seem to retain a sane relationship. Preliminary tests show that the PML as implemented isn't quite perfectly matched: there are small reflections off of the PEC boundaries, and it's not exactly clear what the issue is—though it may be related to the previously mentioned issue. Those issues aside, there are a few important details to discuss regarding the experiments. Each test that was performed used a Ricker wavelet to provide the magnetic field with a source pulse without introducing a DC offset to the field's spectrum. In the spirit of making the source function's behavior scale invariant, some analysis is performed to construct an appropriate discrete representation of the Ricker function. A peak-normalized Ricker function was used here, written as

$$\psi(t) = \left(1 - (t-\tau)^2/\sigma^2\right) \exp\left(-\frac{1}{2}\frac{(t-\tau)^2}{\sigma^2}\right), \tag{18}$$

where τ corresponds to the time of the peak response of the function (or rather a delay in when the source turns on), and σ defines the width of the response. Taking the Fourier transform and maximizing the result shows that the Fourier mode that contains the most power has frequency $\nu_{\max} = \sqrt{2\pi\sigma^2}$. Using this result, along with rewriting the continuous parameters t and τ as discrete parameters $t \to n\Delta t$, $\tau \to m\Delta t$, the Ricker wavelet can be written in terms of the Courant number S_c, the number of points per peak wavelength N_p, and the discrete parameters n, m as

$$\psi_n = \left(1 - 2\pi\frac{S_c^2(n-m)^2}{N_p^2}\right) \exp\left(-\pi(n-m)^2 S_c^2\right). \tag{19}$$

For each simulation performed, ten points were used per wavelength (in order to minimize aliasing artifacts) and an offset of 80 integrations was used (to minimize numerical transients).

The intended use of this simulation was to perform a 2-dimensional beam simulation for a radio antenna; however, technical difficulties (and perhaps a bit too much focus on scaffolding) resulted in this not being realized. As an alternative, various scenarios with dielectric objects and perfect conductors were simulated for a modest grid and timescales

of order one grid crossing. Figure 2 shows three images from a simulation performed with a slab of dielectric material with Gaussian random fluctuations. Figure 3 shows three images from a simulation performed with a dielectric sphere at the center of the simulation grid and a few metal spheres placed nearby the source. In both simulations, the qualitative dynamics of the fields look to be reasonable, with the fields propagating more slowly through the dielectric with deformed wavefronts and wrapping around the metal objects. In order to visualize the fields, the amplitude of the magnetic field scaled by the impedance of the space was used as a normalizing factor at each time step (which provided a good sanity check that, at the very least, the relative strengths of the fields seem to make sense).

V. CONCLUSION

This project provided an interesting, though challenging, introduction into the world of computational electrodynamics. I developed a FDTD simulation Python package for TE_z-polarized fields that employed PML absorbing boundary conditions, complete with a large amount of basic functionality for simulation management that supports linear media and a small set of perfect conductor geometries. The simulation software provides a strong base for developing future, more complex FDTD simulations, though quite a few upgrades will be needed in order to handle the increased complexity. Main features that could be implemented as improvements are extension to TM_z fields and subsequent generalization to three-dimensional fields; better implementation of the PML, in particular the use of a convolutional PML (which also erases the need to consider split fields); extension from linear media to nonlinear media. Additional work is needed to understand the origin of the runaway field amplitudes; however, the simulations seem to retain qualitatively useful information, as the fields retain their proper proportional structure. The code could also do with a wider selection of sources—currently the only source with reasonable performance is the Ricker wavelet (a plane wave function is provided, but I haven't been able to find a

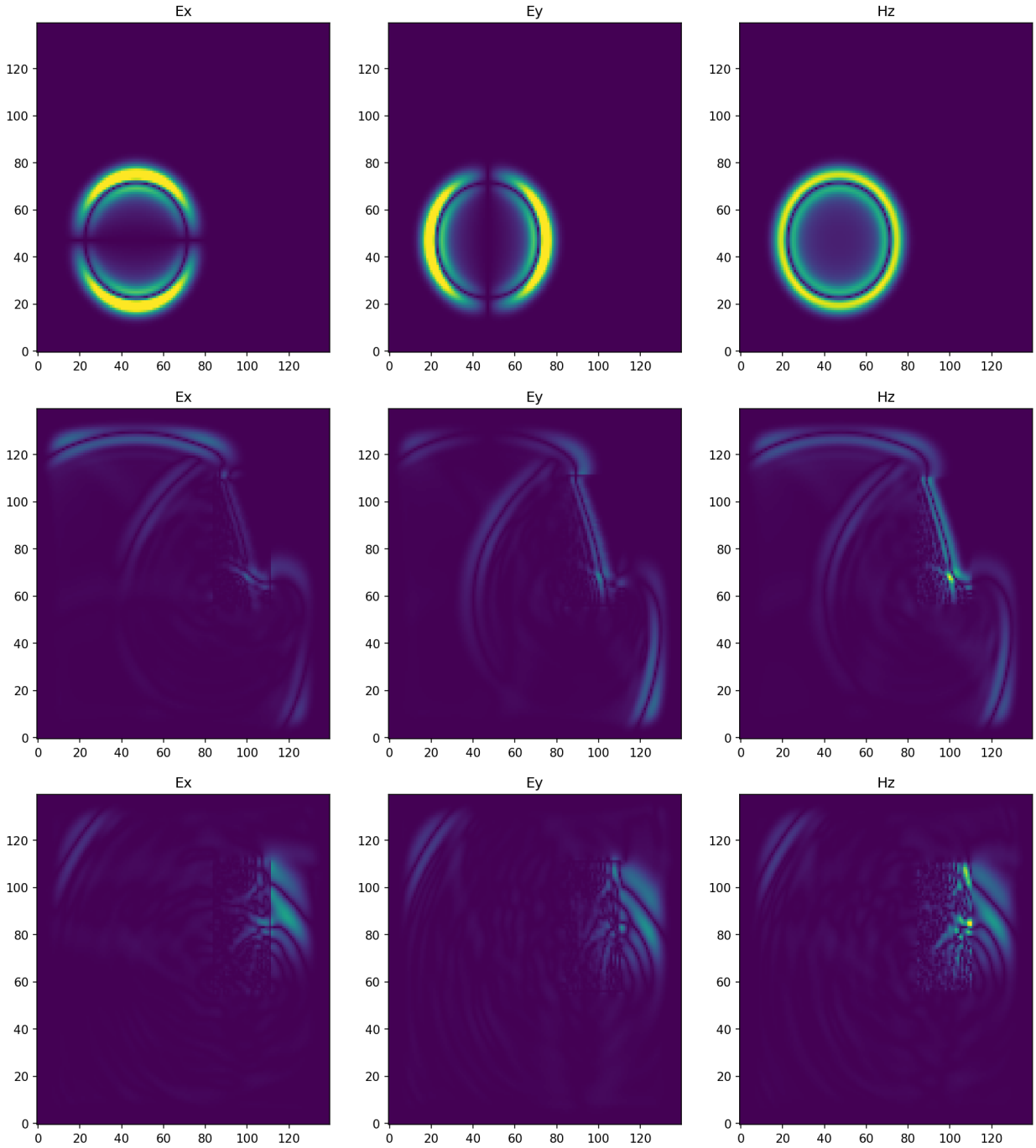

FIG. 2. From left-to-right: E_x, E_y, H_z fields. From top-to-bottom: progression in time, with the first row taken about 90 integrations into the simulation, the second row about 200 integrations in, and the third row around 300 integrations in. This simulation used a rectangular dielectric slab in the upper-right region of the simulation domain with an average relative permittivity of 5 and mean-zero Gaussian random variations with an amplitude of 10%.

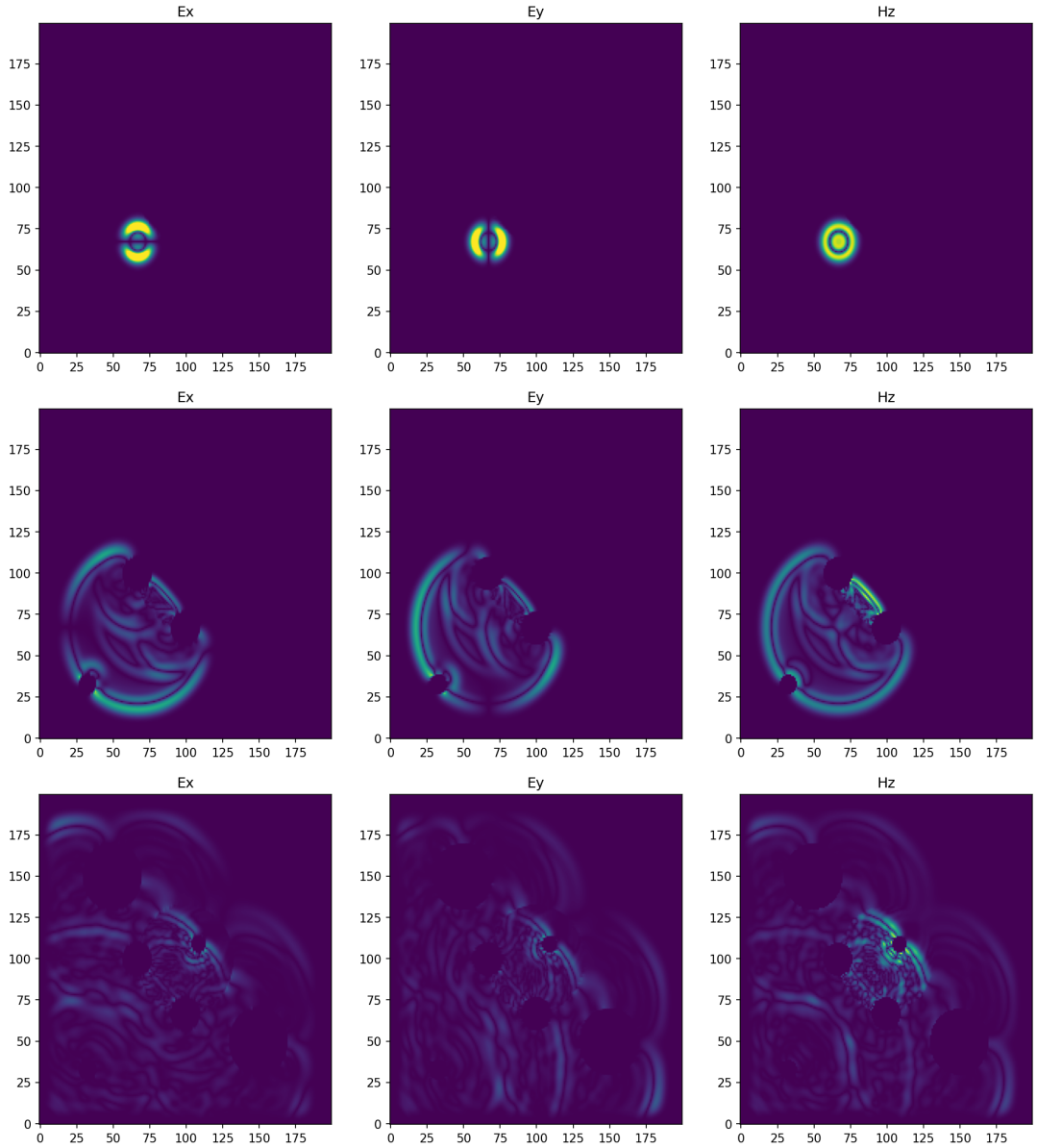

FIG. 3. The left-to-right and top-to-bottom layout is the same as in Figure 2, but with different choices of integrations shown. The top row is about 70 integrations in; the middle row is about 150 integrations in; the bottom row is about 300 integrations in. This simulation consisted of a dielectric sphere at the center of the simulation grid, with three metal spheres embedded in it, as well as three additional spheres placed in all but the upper-right corner of the box. The spheres vary in size, though the second and third rows of plots should suffice for getting an idea of how big each metal sphere is. A Ricker wavelet with ten points per peak wavelength, optimal choice of Courant number, and onset 80 integrations into the simulation was used as a source; it was placed at one third of the way into the box, as measured from the left and from below.

stable set of parameters to use with it).

---

[1] Astropy Collaboration, Price-Whelan, A. M., Sipőcz, B. M., et al. 2018, , 156, 123, doi: 10.3847/1538-3881/aabc4f

[2] Berenger, J.-P. 1994, Journal of Computational Physics, 114, 185, doi: https://doi.org/10.1006/jcph.1994.1159

[3] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. 2020, Nature, 585, 357, doi: 10.1038/s41586-020-2649-2

[4] Schneider, J. B. 2010, Understanding the Finite-Difference Time-Domain Method. www.eecs.wsu.edu/~schneidj/ufdtd